

1. Why Residual Connections Help Optimization

Standard deep convolutional neural networks often suffer from the degradation problem. As depth increases, accuracy saturates and then degrades rapidly. This is not caused by overfitting, but by the fact that deeper networks are inherently harder to optimize. Residual connections solve this by forcing the network to learn a residual mapping, $\mathbf{F}(\mathbf{x})=\mathbf{H}(\mathbf{x})-\mathbf{x}$, rather than the original unreferenced mapping $\mathbf{H}(\mathbf{x})$. The original mapping $\mathbf{H}(\mathbf{x})=\mathbf{F}(\mathbf{x})+\mathbf{x}$. Mathematically, it is much easier for the optimizer to push the residual weights to zero (*leaving the identity mapping $\mathbf{x}$*) than to learn an identity transformation from scratch across multiple non-linear layers. This smoother optimization landscape allows the network to converge faster and achieve lower training error.

2. How Skip Connections Improve Gradient Flow :

The most significant bottleneck in training deep networks is the vanishing gradient problem. During backpropagation, the network uses the Chain Rule to calculate how much to update the weights in the early layers. the gradient from the loss function $\mathcal{E}$ is multiplied by the weights and the derivative of the activation function at every single layer as follows:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L} \prod_{i=l}^{L-1} w_i \cdot f'(x_i)$$

Gradients are continually multiplied by small weights; in standard networks, they eventually become so small that early layers stop learning. Skip connections provide a direct, uninterrupted path for the gradient to flow backward. Because the output of a residual block is $\mathbf{F}(\mathbf{x}) + \mathbf{x}$, the derivative with respect to $\mathbf{x}$ will always include a **+1** term (*derivative of $\mathbf{x}$ with respect to $\mathbf{x}$ is 1*). This guarantees that the gradient never vanishes completely, ensuring that even the very first convolutional layers receives information and never stops learning.

3. The Effect of Network Depth on Performance

Network depth is crucial for hierarchical feature extraction. Early

layers learn edges and textures, while deeper layers learn abstract concepts like tires or wings. In a traditional CNN, adding depth eventually damages performance due to degradation. However, by utilizing residual blocks, we successfully decoupled depth from degradation. Our 3 stage architecture allowed the model to build complex, high-level representations of the 32x32 CIFAR10 images, directly translating network depth into a highly generalized validation accuracy of nearly 88% without overfitting.

4. Challenges Encountered During Implementation

Dimension Mismatches: When transitioning between stages, the spatial dimensions halved while the channel depths doubled. A standard skip connection $\mathbf{x}$ could not be added to $\mathbf{F}(\mathbf{x})$ due to differing tensor shapes. This was solved by implementing an automated 1×1 convolutional projection layer within the shortcut path whenever $\text{stride} \neq 1$ or $\text{in_channels} \neq \text{out_channels}$.

Data Pipeline Bottlenecks: Heavy network throttling from external dataset host servers (like the University of Toronto) initially prevented data loading. This was overcome by manually downloading the raw archive, extracting it locally, and mounting it as a native cloud dataset to bypass network latency and read-only file system restrictions.

Root Pathing: PyTorch's CIFAR10 loader expects a specific parent directory structure. Directly pointing the root argument to the inner cifar-10-batches-py folder caused corruption errors, which required redirecting the path to the parent directory to allow PyTorch to locate the batches naturally.

5. Architecture Choices

To meet the specific constraints of the task, the architecture was designed with the following parameters:

- **Stages & Blocks:** The network utilizes exactly 3 residual stages, with each stage containing 2 residual blocks.
- **Channel Depth:** The channel depth doubles at each stage progression (16→32→64) to capture increasingly complex feature maps as the spatial dimensions are reduced.

- **Downsampling:** Spatial downsampling is handled using a stride of 2 in the first block of Stages 2 and 3.
- **Final Layer:** A Global Average Pooling (GAP) layer is utilized before the final fully connected classifier, drastically reducing the parameter count and preventing overfitting compared to flattening standard dense layers.